
Utvikling av relativt nivå hos maskinlæringsmodeller trent med selvspill

Trygve Jørgensen

Norges Teknisk-Naturvitenskapelige Universitet

tryggjor@stud.ntnu.no

Sammendrag

I konkurransebaserte multiagent-miljøer er stabil trening av modeller ikke trivielt. Selvspill er i dag en av de mest sentrale metodikkene brukt og har tillatt maskinlærings-modeller å oppnå supermenneskelige evner uten initielle forkunnskaper om sitt miljø. Det er bredt at akseptert at mer trening resulterer i bedre modeller, men kartlegging av relativ forbedring og identifisering av strategier varierer derimot betydelig mellom miljøer. Dette gjør det utfordrende å måle effekten av selvspill.

I denne rapporten ble det utviklet et egendefinert nullsumspill-miljø hvor en Proximal Policy Optimization (PPO) algoritme trente modeller utsatt for selvspill. Modellene ble periodisk lagret gjennom treningsprosessen og i ettertid evaluert opp mot sine tidligere kopier. Ved bruk av diverse visualiseringmetoder ble modellenes relative nivå og strategier dokumentert, og den faktiske stabiliteten til treningsprosessen ble grundig drøftet.

1 Introduksjon

1.1 Forsterkende læring

Forsterkende læring (RL) er i dag en av de mest sentrale konseptene innen maskinlæring (ML). Skal en trene modeller i dynamiske miljø uten eksisterende treningsdata, eller utforske nye strategier i uløste spill er RL en ekstrem generaliserbar og ofte effektiv metode. I RL kontrollerer ML-modellen en eller flere agenter som gjennom prøving og feiling generer dataene den selv blir trent på. Selv med dagens beste algoritmer er trening av modeller på ingen måte trivielt, og konvergering til et globalt maksimum kan sjelden garanteres¹. I mange tilfeller er RL likevel metodikken som resulterer i best mulig ytelse (se 2.2).

1.2 Selvspill

Selvspill er oftest brukt i konkurransebaserte nullsumspill-miljøer og innebærer at en modell spiller mot en kopi av seg selv. Modellen kan dermed alltid bli utsatt for strategier den selv har oppdaget, og hvis den samtidig blir periodisk eksponert for eldre taktikker, vil den i teorien utvikle en robust og fullstendig strategi. Det er likevel gitt at en modell alltid vil bli bedre med tid eller hvordan den faktisk lærer nye strategier fra gjentatt feiling.

1.3 Problemstilling

Formålet med rapporten er å kartlegge modeller relative utvikling ved å evaluere tidsbestemte kopier opp mot hverandre og identifisere viktige momenter i treningsprosessen. Hvor effektiv er modellen på å motvirke sine tidligere kopier? Blir modellen alltid bedre? Hvor stor påvirkning har utvalget av modellens motstandere på dens utvikling? Gjennom analyse av resultatene skal det utforskes hva slags konklusjoner en kan dra rundt treningen i det gitte miljøet, samt å drøfte eventuelle forbedringer for å ytterligere stabilisere læringsprosessen.

¹Ilyas et al. (2020)

2 Relatert Arbeid

2.1 Proximal Policy Optimization

PPO-algoritmen ble oppfunnet i 2017 og er i dag sett på som en av de beste algoritmene for RL. PPO er en direkte etterkommer av Trust Region Policy Optimization² og er både utviklet av og brukt som standard hos OpenAI³. Den er designet for å balansere en rekke faktorer som skal gjøre den enkel å implementere, lett å justere og effektiv å trene.

Grovt oppsummert oppnår PPO dette med et klippet $\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)$ "surrogat-objektiv" $r_t(\theta)\hat{A}_t$ summert over en gruppe steg $\hat{\mathbb{E}}_t$ som resulterer i tapsfunksjonen L^{CLIP} utnyttet for gradnedstigning. Surrogat-objektivet er produktet av modellens selvsikkerhet $r_t(\theta)$ som en funksjon av sannsynligheten til det gjeldene nettverket over sannsynligheten fra forrige nettverket, og modellens estimerte fremtidige belønning $\hat{A}_t$, for et gitt steg t . Surrogat-objektivet skifter modellens læringsfokus fra absolutte belønninger til modellens forventet ytelse. Basert på modellens selvsikkerhet vil PPO med klipping begrense steglengden i positiv utvikling, men aggressivt korrigere for dårlig oppførsel.

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t \right) \right]$$

Selvsikkerhet	$r_t(\theta)$ verdi	Prestering	$\hat{A}_t$ verdi	Tapsfunksjon	L^{CLIP} verdi
Lav	$r_t(\theta) < (1 - \epsilon)$	Dårligere enn forventet	$\hat{A}_t < 0$	Negativ klippverdi	$(1 - \epsilon)\hat{A}_t$
Lav	$r_t(\theta) < (1 - \epsilon)$	Bedre enn forventet	$\hat{A}_t > 0$	Surrogat objektiv	$r_t(\theta)\hat{A}_t$
Høy	$r_t(\theta) > (1 + \epsilon)$	Dårligere enn forventet	$\hat{A}_t < 0$	Surrogat objektiv	$r_t(\theta)\hat{A}_t$
Høy	$r_t(\theta) > (1 + \epsilon)$	Bedre enn forventet	$\hat{A}_t > 0$	Positiv klippverdi	$(1 + \epsilon)\hat{A}_t$

Table 1: L^{CLIP} verdi basert på $r_t(\theta)$ og $\hat{A}_t$

2.2 Selvspillings nivåtak

Selvspill har fra sin første implementering i 1959 illustrert at en modell kan bli bedre enn mennesket som selv programmerte miljøet⁴. I 2017 ble DeepMinds AlphaGo Zero⁵ et av de meste definitive eksemplene på at en modell, trent eksklusivt mot seg selv, kan oppnå supermenneskelige ferdigheter. OpenAIs Five⁶ beviste to år senere at selvspill med PPO-algoritmen også kan utkonkurrere verdensmestere, denne gangen i det lagsbaserte videospillet Dota 2.

2.3 Analyse av selvspillingsprosedyrer

I OpenAIs dypdykk i konkurransebaserte multiagent-miljøer⁷ ble det studert hvilket påvirkninger faktorer som tilfeldige miljøparametre og motstandervalg hadde på treningsdynamikken. Her ble vinnraten av metodikkene grundig kartlagt, og det ble blant annet identifisert at modeller eksklusivt trent mot sin siste kopi utsettes for grov overtilpasning og vinnraten var oftest best hos modeller utsatt for forskjellige inntrykk. For mye variasjon kan samtidig betydelig stagnere tidlig læring.

2.4 Mangel på analyse av relativ modellutvikling

Det er gjort mye omfattende arbeid rundt kartlegging av forskjeller til modeller trent med forskjellige prosedyrer og hyperparametre. I selvspilling er det fortsatt noe mindre grundig analyse gjort på modellers relative ytelse mot sine tidligere kopier. Det er mulig å anta at det fortsatt er nye oppdagelser som

²Schulman et al. (2017a)

³Schulman et al. (2017b)

⁴Samuel (1959)

⁵Silver et al. (2017)

⁶OpenAI et al. (2019)

⁷Bansal et al. (2018)

kan avdekkes ved nøyere gransking av hvordan modeller faktisk lærer og eventuelt glemmer strategier når den eksklusivt er utsatt for selvspill.

3 Metode

I denne oppgaven er det utnyttet Gymnasium-biblioteket som rammeverk for utvikling av et egendefinert selvspill-miljø og Stable-Baselines3s (SB3) PPO-algoritme for trening av modellen.

3.1 Miljø

Miljøet utviklet er et fysikkorientert nullsumspill og kan anses som en grov, todimensjonal forenkling av sporten volleyball eller badminton. Hver agent er en flyvende kule som kan vilkårlig bevege seg på sin side av banen. Selve ballen er påvirket av tyngdekraft og kan reflekteres basert på innfallsvinkel ved kontakt med en agent. Ved episodens oppstart starter agentene midt i hver sin kvadrant og ballen får en maksimal fartsvektor med en tilfeldig retning mot en tilfeldig motstander. Retningen er alltid slik at agenten skal kunne returnere ballen hvis spilt riktig.

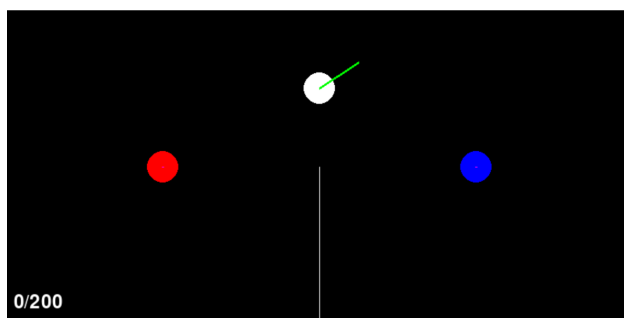


Figure 1: Miljøet ved oppstart. Observert: venstre agent(rød kule), høyre agent(blå kule), ball med fartsvektor(hvit kule og gul strek) og nett(hvit sentrert strek)

Reflekterer en agent ballen slik at den berører motstanderens bakke, vil agenten vinne. Skulle ballen berøre tak, vegg eller nett vil siste agent som berørte ballen tape og episoden termineres. Har 200 steg passert uten terminering tvangstoppes episoden og agenten med ballen i sin banehalvdel taper. For opprettholde et sant nullsumspill og eliminere eventuelle utnyttelser av miljøet belønnes agenten ± 1 basert på om den vinner eller taper, uavhengig om episoden termineres eller tvangstoppes.

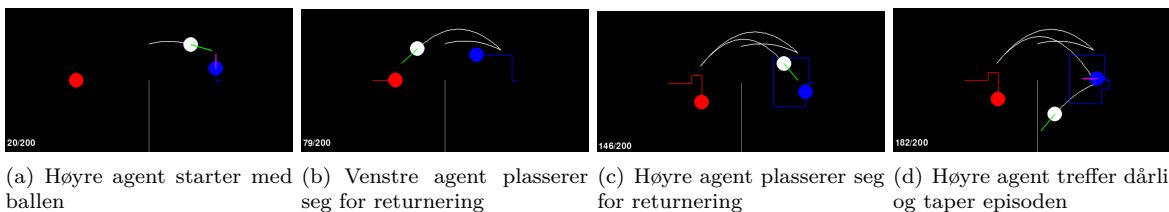


Figure 2: Eksempel på en vilkårlig episode i miljøet.

3.1.1 Handling

Hver agent har et handlingsrom på 5 diskrete handlinger. Hver bevegelse skjer øyeblikkelig mellom steg og kan sees på som en translasjon med konstant fart. Bevegelsen valgt blir lagret som agentens fart for gitt steg og inkluderes i eventuell utregning av reflektering mellom ball og agent. Størrelsen på ballens reflekterte fartsvektor vil derfor være avhengig av ballens og agentens relative fart.

Handling	Bevegelse
0	Ingen
1	Venstre
2	Høyre
3	Opp
4	Ned

Table 2: Agentens handlingsrom

3.1.2 Observasjon

Observasjonsrommet er på 9 dimensjoner og gir agentene nær perfekt informasjon. Agentenes fart er ekskludert ettersom de ikke er påvirket av akselerasjon og predikering av motstanderens neste posisjon er dermed umulig⁸. Dette reduserer variablene agentene må håndtere og forenkler læringsprosessen.

Beskrivelse	Min	Max
Ball x	0	2
Ball y	0	1
Ball fart x	-0.015	0.015
Ball fart y	-0.015	0.015
Venstre agent x	0	1
Venstre agent y	0	1
Høyre agent x	1	2
Høyre agent y	0	1
Tid	0	1

Table 3: Agentens observasjonsrom

3.1.3 Motstander og speilvendt-het

Gymnasiums rammeverk støtter i utgangspunktet ikke selvspill. Miljøet ble derfor en gitt en motstanderattributt som ved initialisering blir satt til en kopi av ML-modellen som skal trenes. Motstanderattributten kan så vilkårlig oppdateres til en ny kopi når oppdatering av motstander ønskes. Ettersom miljøet er totalt symmetrisk langs nettet, har miljøet metoder for speilvendte observasjoner og handlinger. Modeller kan da eksklusivt trenes som venstre agenter, og samtidig utnyttes som høyre motstandere uten å være påvirket av sideskiftet, ettersom de mottar speilvendte observeringer som tilsier at de fortsatt er på venstre side.

3.1.4 Selvspill

Utskifting av motstander skjer hvert 40000 steg som tilsvarer på det minste 200 episoder om alle episoder resulterer i tvangsstopp. Dette ble valgt med utgangspunkt i andre veletablerte maskinlæringsbiblioteker⁹ hvor det anbefales å bruke mellom 10000-100000 steg.

For å unngå overtilpasning av gjeldene strategier ble 20% av treningen gjennomført mot en tilfeldig, tidligere kopi valgt med eksponentiell vektning mot de senere kopiene. Dette er en grov etterligning av metodikken brukt for treningen av OpenAI Five.

Det ble gjennomført tre varierte treningsrunder for å undersøke effekten utvalg av motstandere hadde på treningsdynamikken. Modellene som skulle evalueres ble kopiert hver 1×10^6 steg uavhengig av nylig prestasjon. 1×10^6 steg representerer ca 5 og 1/2 time med simulert trening som på en Intel i7-7700 CPU med 4 parallelle miljøer tok ca 20 minutter.

⁸...om man ignorerer muligheten for læring av motstanders fullstendige strategi basert dens fartsvektor i et visst tidspunkt

⁹ML-Agents

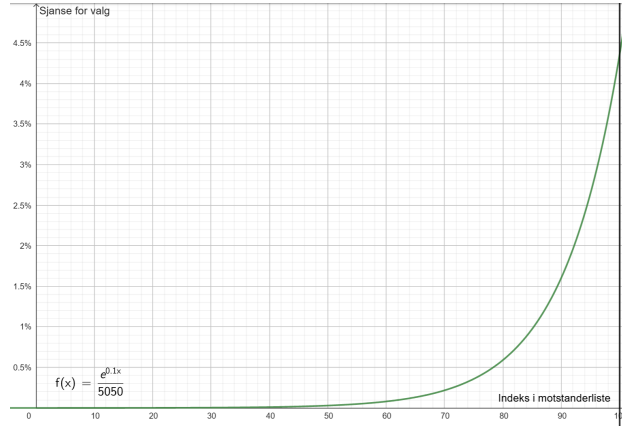


Figure 3: Sannsynligheten for valgt motstander i ved antall motstandere $l = 100$ og indeks $= x$ gitt ved $\frac{e^{0.1 \times x}}{\sum_{n=1}^l n}$

Navn	Runder trent	Vinnrate vindu	Ytelseskrav for motstander	Tilfeldig motstandervalg
Benchmark	23×10^6	1000 episoder	60% vinnrate	Eksponentielt fordelt
Hardest	24×10^6	200 episoder	75% vinnrate	Eksponentielt fordelt
Generalized	42×10^6	Ingen vinnrate	Ingen ytelseskrav	Uniformt fordelt

Table 4: Treningsrundene gjennomført med og uten ytelseskrav for motstandere

3.2 Evaluering

Kartlegging av nivåforskjell mellom modeller ble utført med deterministiske startbetingelser for ballens fartsvektor og episodens startside. Modellene spilte 2 sett på 1000 episoder hver, der hvert sett ble spilt med modellene på hver sin side for å sikre likesidet eksponering av eventuelle fordeler og ulemper grunnet miljøets startbetingelser. Siden både startbetingelsene og handlingene var deterministiske, gav identiske simuleringer alltid samme resultat.

3.2.1 Visualisering av vinnrate

For å enkelt identifisere nivåforskjeller mellom modeller ble resultatene fra evalueringen plottet på et todimensjonalt rutenett hvor den vertikale akse består av den venstre modellens vinnrate ovenfor den høyre modellen som ligger langs den horisontale akse. Dette viser modellenes relative nivå om hverandre og illustrerer om resultatene viser faktiske nivåforskjeller eller om de kan ansees som unntak grunnet "uheldige" startbetingelser.

Samlede linjediagram ble også utnyttet for å identifisere eventuelle modeller som presterte særdeles dårlig mot en større andel motstandere. Hver linje er en venstre modell og dens farge tilsier antall steg trent. Den horisontale akse representerer høyre motstander treningssteg og den vertikale akse er antall seiere for venstre modell i gitt sett.

3.2.2 Visualisering av strategi

Situasjonspesifikke strategier er både vanskelig å observere og representerer ikke nødvendig modellens overordnede oppførsel. Istedenfor er det her utnyttet varmekart over agentens posisjon under evaluerings-episodene. Forskjeller i agentens overordnede bevegelse blir dermed lettere å identifisere og kan brukes rundt drøfting av strategier.

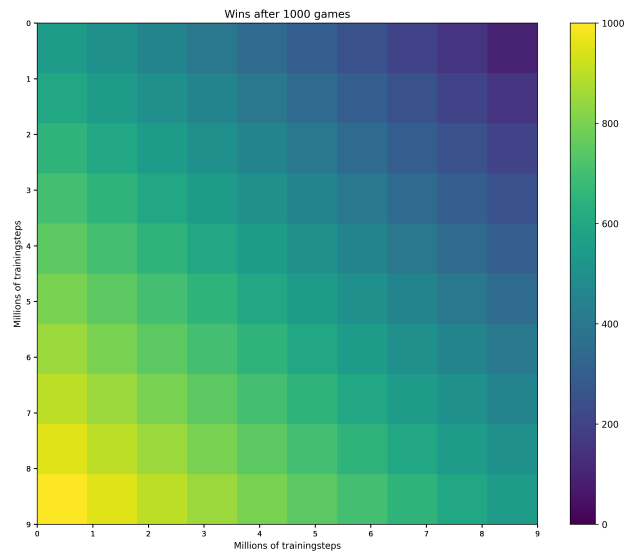


Figure 4: Visualisering med rutenett av fiktiv modell med lineær ferdighetsutvikling

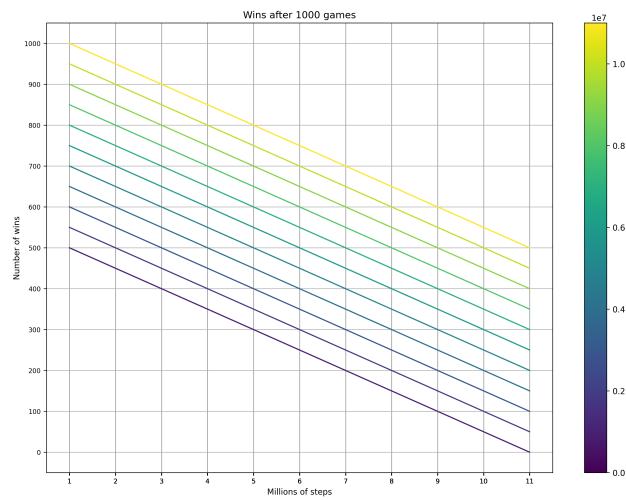


Figure 5: Visualisering med linjediagram av fiktiv modell med lineær ferdighetsutvikling

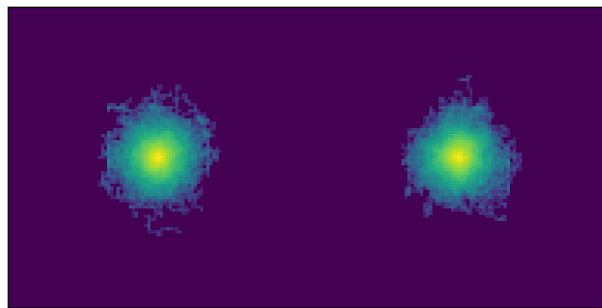


Figure 6: Varmekart av agenter kontrollert av tilfeldig handlinger over 1000 runder

4 Resultater og Diskusjon

4.1 Vinnrate

Samtlige treningsrunder har illustrert at modellene generelt sett blir bedre med tid, men med en støyete utvikling. I Generalized ser man også modellene divergere betydelig etter 32×10^6 steg. Resultatene er bredt sett grunnet faktiske nivåforskjeller ettersom uregelmessige avvik er speilet langs den diagonale akse. Eksempelvis i Benchmark utkonkurrerer modellene trent 1×10^6 og 2×10^6 steg flere av de mer erfarne modellene, uansett hvilken side de spiller på.

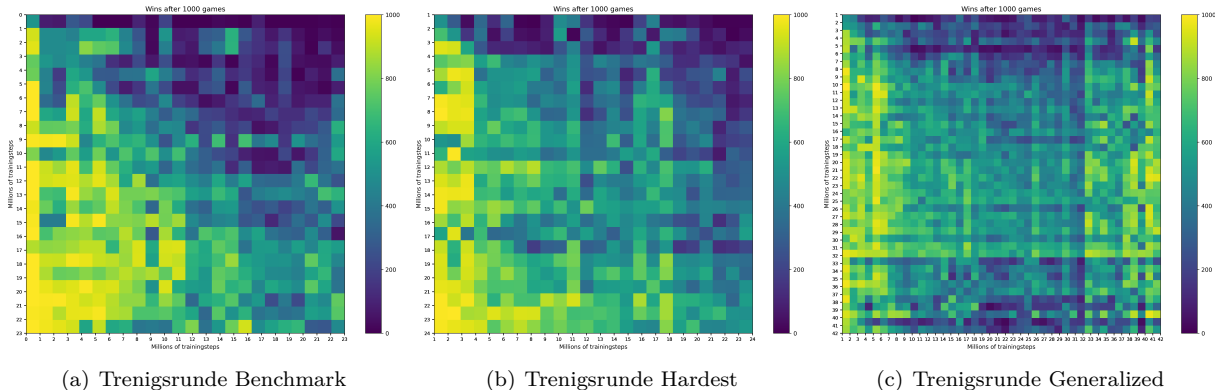


Figure 7: Visualisering av vinnrate med rutenett

Utifra dataene kan det ikke dras sikre konklusjoner rundt påvirkningen ytelseskrav hadde på treningsrundene ettersom divergeringen observert i Generalized skjedde ca 10×10^6 steg etter Benchmark og Hardest stoppet trening. I linjediagrammet til Generalized blir det tydeligere at modellen trent i 32×10^6 steg kan ansees som treningsrundens mest robuste modell før dens etterkommere drastisk divergerer.

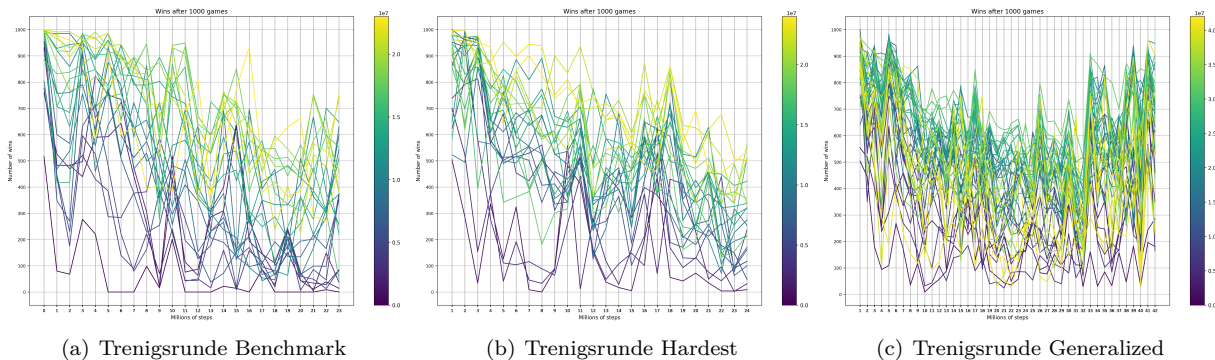


Figure 8: Visualisering av vinnrate med linjediagram

4.2 L ring og tap av strategier

Modellenes utvikling av strategier og spillforst else kan klart observeres i deres posisjonelle varmekart. I starten av treningsrunden ser vi stor forskjell i strategier og det svinger mellom aggressivt og konservativt spill. N r modellen over tid utvikler mer robust ballforst else kan vi tydelig se at strategiene konvergerer til   posisjoneres seg s  n rt nettet mulig.

I Benchmark presterte modellen 15×10^6 s rdeles d rligere enn en rekke tidligere modeller. Spesielt hadde modellene 1×10^6 og 2×10^6 en unormalt h y vinnrate i dette tilfellet. Dette er h yst sannsynlig et tidlig

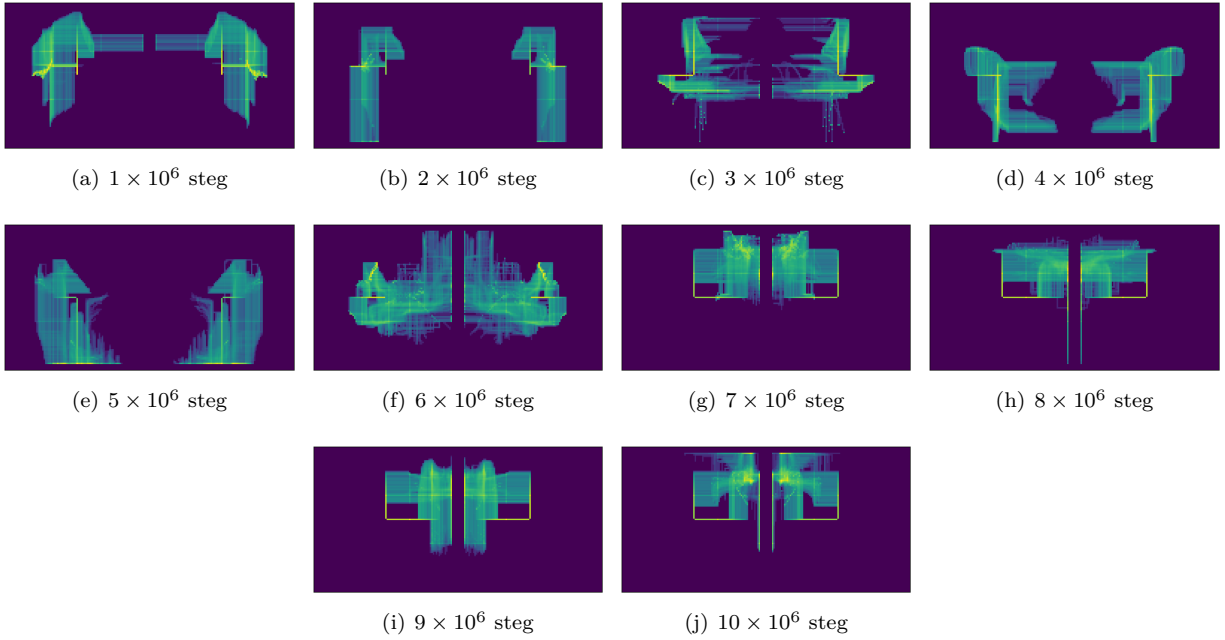


Figure 9: Varmekart til de ti første evaluerte modellene i Benchmark

eksempel på overtilpasning senere modeller i større grad behersker "tett" spill, noe modellen 15×10^6 har lært å motvirke, på bekostning av å glemme hvordan man responderer til "enklere" spill. Dette spesielt tydelig mot 3×10^6 som viser en aggressiv spillestil, men med over 99% tap mot 15×10^6 .

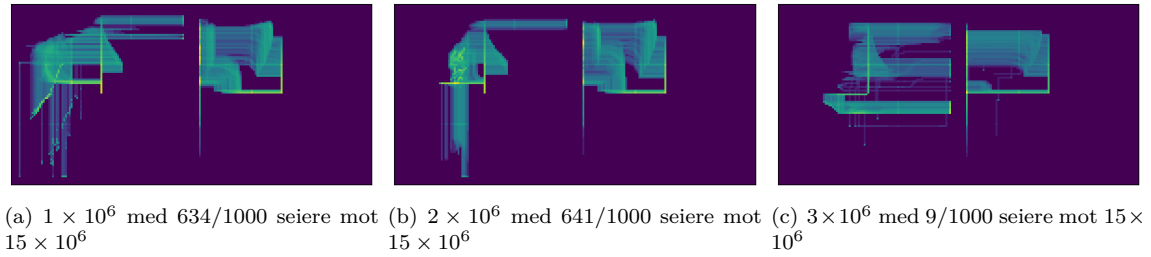


Figure 10: Varmekart til av evaluerings-rundene for enkelte modeller i Benchmark

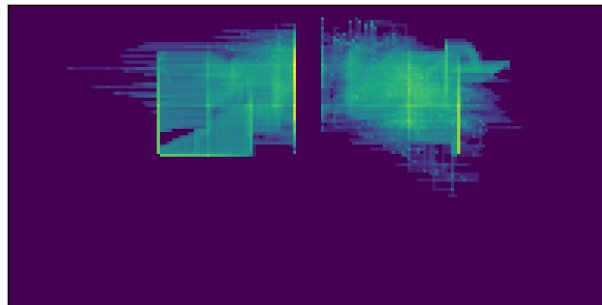


Figure 11: Generalized 32×10^6 med 900/1000 seiere mot 33×10^6

I divergeringen observert i Generalized mellom 32×10^6 og 33×10^6 er det bare nyanserte strategiforskjeller til tross for en enorm nivåforskjell. 32×10^6 viser klar bevegelse, primært inntil nettet, mens 33×10^6 , som generelt beveger seg innenfor det samme området, har tydelige sporadiske bevegelser.

Grunnlaget for hvorfor divergeringen etter 33×10^6 tok plass kan ikke begrunnes med fullstendig sikkerhet. Generalized tilfeldige motstandere ble valgt uten ytelseskrav og det er mulig at 33×10^6 ble et bristepunkt hvor modellen forsøkte å unødvendig tilpasse seg for objektivt dårlige motstandere, og derav miste fundamentet den tidligere hadde.

4.3 Feilkilder

Resultatene i denne rapporten har liten verdi for faktisk utførelse av effektiv selvspill. Påvirkningen av PPO-algoritmens hyperparametere ble aldri testet, så det er høyest sannsynlig at treningen ytterligere stabiliseres med gjennomtenkt justering av parametre. SB3 standard nevralt nettverk består av 2 skjulte lag på 64 noder hver, hvor effekten av forskjellige nettverk heller ikke ble testet. Det er godt mulig at til tross miljøets "primitive" dimensjonalitet, er ikke nettverket komplekst nok for å beherske universalt robuste strategier. Hadde modellene blitt trent betydelig lengre er det også godt mulig at nivåutvikling villet illustrert flere viktige trender.

Modellens kalkulerter vinnrate for ytelseskravet er også en potensiell feilkilde ettersom vinnratens vindu ble kalkulert fortløpende istedenfor forhåndsbestemte evalueringrunder slik evalueringen ble gjennomført. Det er likevel

5 Konklusjon og Videre Arbeid

I denne rapporten er det illustrert hvordan modeller trent med selvspill i et nullsumspill-miljø lærer konkrete strategier som konservativt og aggressivt spill, og hvordan de "glemmer" strategier som kan resultere i både midlertidig og permanent divergering. Det er fortsatt et stort behov for å videre utføre lignende eksperimenter med ytterligere systematisk datainnsamling som vil bedre kartlegge selvspillings effekt på modellens treningsdynamikk, som igjen kan avdekke nye forbedringer rundt effektivisering av selvspill.

References

- Trapit Bansal, Jakub Pachocki, Szymon Sidor, Ilya Sutskever, and Igor Mordatch. Emergent complexity via multi-agent competition, 2018. URL <https://arxiv.org/abs/1710.03748>.
- Andrew Ilyas, Logan Engstrom, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. A closer look at deep policy gradients, 2020. URL <https://arxiv.org/abs/1811.02553>.
- ML-Agents. Unity ml-agents toolkit, 2024. URL <https://github.com/Unity-Technologies/ml-agents/blame/89a6357016f4bd685bb0004f83a3a1c2b27a0e05/docs/Training-Configuration-File.md#L196>. Open-source software with multiple contributors.
- OpenAI, :, Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Dębiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, Rafal Józefowicz, Scott Gray, Catherine Olsson, Jakub Pachocki, Michael Petrov, Henrique P. d. O. Pinto, Jonathan Raiman, Tim Salimans, Jeremy Schlatter, Jonas Schneider, Szymon Sidor, Ilya Sutskever, Jie Tang, Filip Wolski, and Susan Zhang. Dota 2 with large scale deep reinforcement learning, 2019. URL <https://arxiv.org/abs/1912.06680>.
- A. L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959. doi: 10.1147/rd.33.0210.
- John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. Trust region policy optimization, 2017a. URL <https://arxiv.org/abs/1502.05477>.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017b. URL <https://arxiv.org/abs/1707.06347>.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy P. Lillicrap, Fan Hui, L. Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 550:354–359, 2017. URL <https://api.semanticscholar.org/CorpusID:205261034>.